MR Risk Assistant

AI-assisted merge request risk triage for code review

Final Report | MBA ZG583 Management of AI Products

TEAM018 | M Mahadev (2025mb26043)

October 2026

Prototype and sources: github.com/MAIP2026July/TEAM018

01 Executive Summary

MR Risk Assistant: AI-assisted merge request risk triage for code review TEAM018 | M Mahadev (2025mb26043) | MBA ZG583 Management of AI Products

---

Problem

In most engineering teams, every merge request (MR) goes through roughly the same review, no matter what it changes. A typo fix in a README and a change to login logic sit in the same queue and often get the same 10-minute look. Reviewers are busy, so in practice the queue is worked oldest-first or by whoever shouts loudest.

The result is that review effort is spread evenly while risk is not. The changes that actually hurt us (a leaked key, an unsafe Kubernetes setting, a payment calculation without tests, a migration that locks a table) usually look small and pass without anyone senior looking at them. Large but harmless MRs, like UI refactors or translation updates, get the extra attention instead.

Target user

- **Primary:** the reviewer / tech lead who has 15-30 open MRs and has to decide where to spend their limited time.
- **Secondary:** the engineering manager who decides who reviews what, and the security / platform team who should be pulled in for specific kinds of change.

Product concept

A review assistant that sits in the MR workflow. For each MR it:

1. reads the diff and metadata (paths, size, test changes, risky patterns),
2. gives a risk score and a Low / Medium / High band,
3. explains *why* in plain language,
4. suggests a review path (standard, senior, security, infra, add tests),
5. lets the reviewer accept or override, with a reason.

It does **not** approve or block merges. The human reviewer stays accountable.

Why AI?

Static rules (CODEOWNERS, "big MR = careful review") already exist and they help, but they miss what matters. In my sample set, the "look harder at MRs over 300 lines" rule caught only 1 of 11 problem MRs and 4 of its 5 flags were false alarms. Risk depends on a *combination* of signals (where the change is, whether tests changed, what patterns appear) and the right weights differ between codebases. That is a learning problem. Separately, an LLM is good at the part rules are bad at: reading a diff and writing a short, specific explanation a reviewer can act on.

So the design is a **hybrid**: deterministic scanners and rules for things that must never be missed (secrets, privileged containers), an ML risk score learned from our own MR history, and an LLM only for the explanation.

Recommendation

Prototype done, proceed to a narrow pilot. Do not scale yet.

Pilot with one team / 2-3 repositories for 8 weeks in "shadow then advisory" mode. Move forward only if the proof gates in the roadmap are met: it catches more problem MRs than the current process without flagging more than ~30% of the queue, and reviewers keep the override rate below ~25%. If reviewers ignore it or it floods them with false alarms, stop.

The prototype (prototype/index.html) shows the workflow end to end on 24 synthetic MRs. It proves the interaction and the trust design. It does not prove real-world accuracy, because I wrote both the rules and the test data.

02 Product Strategy and Opportunity Framing

Product vision

Every reviewer should know, within a few seconds of opening the queue, which changes deserve their full attention and why, so that review time goes where the risk actually is.

JTBD

Reviewer / tech lead

> When I have a long list of open MRs and limited time, I want to quickly see which ones are risky and what exactly to look at, so that I can spend my review time where a mistake would hurt the most, and not miss something serious because it looked small.

Engineering manager

> When I am planning review capacity and who reviews what, I want to see where risky changes are coming in, so that I can pull in security or senior reviewers before an incident rather than after.

Opportunity framing

Why this problem matters

- Code review is one of the most expensive recurring activities in an engineering team, and it mostly uses senior people's time.
- The most costly problems (leaked credentials, insecure config, broken migrations, payment bugs) often come from *small* changes. They do not look scary, so they get a light review.
- After an incident, the post-mortem usually finds that "the MR was approved, nobody noticed". That is a triage failure, not a skills failure.

Why current alternatives are weak

Current approach	What it does well	Where it falls short
Review everything equally	Simple, fair	Wastes senior time on trivial MRs and spreads attention thin on risky ones
"Big MR = careful review"	Easy to apply	Size is a weak signal; small risky changes slip through
CODEOWNERS / required approvers	Routes by folder	Static. Says who must approve, not how risky the change is
SAST / secret scanners in CI	Very good at known patterns	Output is a long list of findings, not a review priority. Many teams ignore the noise
AI code review bots	Comment on code style and bugs line by line	Add more comments to every MR. They don't answer "which MRs need my attention first"

Why AI may help

- The risk of an MR depends on a mix of signals, and the right weights are specific to each codebase. A model trained on our own MR history (which MRs led to reverts, hotfixes, incidents) can learn this better than hand-written rules.
- An LLM can turn the signals plus the diff into a short explanation that a reviewer can check in 20 seconds. That is what makes the score usable.

Opportunity matrix

I considered several possible features and scored them on business value and feasibility for a first version.

Idea	Business value	Feasibility	Priority
Risk score + band per MR with reasons	High	High	**MVP**
Suggested review path (security / infra / senior / tests)	High	High	**MVP**
Reviewer override with reason capture	High (trust + learning data)	High	**MVP**
Secret / unsafe-config detection (reuse existing scanners)	High	High	**MVP**
LLM explanation of the diff	Medium-High	Medium	**Pilot**
Learned ML model on historical MRs	High	Medium (needs labelled history)	**Pilot**
Manager dashboard: risk by repo / service	Medium	High	Later
Auto-assign the right reviewer	Medium	Medium	Later
Blast-radius analysis (who imports this code)	High	Low	Later
Automatically block high-risk merges	Low (hurts trust)	High	**Rejected**

The last row matters. Blocking merges on a model score would be easy to build, but it moves accountability from the reviewer to the tool, and the first false block would kill trust. So I rejected it on purpose.

Build vs buy vs partner/API

Recommended path: reuse existing scanners + LLM via API + build the risk and routing layer ourselves.

Layer	Decision	Why
Secret and SAST detection	**Reuse** open-source scanners (gitleaks / semgrep type tools) already common in CI	Mature, cheap, well understood. No reason to rebuild
Explanation text	**API** to a hosted LLM (or a self-hosted model if code cannot leave the company)	Learning speed matters more than owning a model at this stage
Risk score, routing, override capture	**Build**	This is the part that is specific to our codebase and our process. It is the "judgment layer" and is the actual product
Git platform integration	**Use platform APIs / webhooks** (GitLab / GitHub)	Show the result inside the MR where reviewers already work

Main trade-offs

- An LLM API means code leaves our network. For many companies that is not acceptable. Fallback: self-hosted model, or only send the signal summary and file names, not the full diff.
- Building the scoring layer ourselves means we need labelled history. If we don't have it, the rules version (what the prototype runs) is still useful as a v1.

First wedge

Start narrow:

- **one team, 2-3 backend repositories** that have a mix of auth, payments, data and infra code,
- **only triage**, no auto-comments, no blocking,
- **shadow mode first** (score is logged, not shown) for 2 weeks, so we can compare it with what actually happened without influencing reviewers.

This is small enough to run without a big rollout, and it hits the MRs where the cost of a miss is high.

Course case connection

- **Netflix (metrics and evaluation lens):** the lesson I took is that a model metric is only useful if it is tied to a business outcome. Here, "AUC of the risk model" means nothing on its own. What matters is whether problem MRs get the right review and whether reviewer time goes down on low-risk ones. That is why my evaluation tracks recall of problem MRs *and* share of queue flagged.
- **Bias in lending / Apple Card (ethics lens):** if the model uses author history, it starts scoring people instead of changes. Junior developers would be flagged more, which is exactly the kind of proxy unfairness discussed in class. I left author features out deliberately (see 07).
- **Instacart / Tesla (lifecycle and drift lens):** codebases change all the time. New folders, new frameworks, new team habits. A model trained on last year's MRs will drift, so the roadmap includes retraining and monitoring the override rate as an early drift signal.

03 PM Artifact Pack

Problem statement

Engineering teams review every merge request with roughly the same effort, while the risk of MRs varies a lot. Reviewers have no quick, consistent way to see which MRs carry real risk (security, data, payments, infra, untested logic) and what to check. As a result, small high-risk changes get a light review and incidents follow, while senior time goes into large low-risk MRs.

The decision being improved: for each open MR, *how much review does this need, and from whom?*

Product vision

The reviewer opens the queue and sees it sorted by risk, with a one-line reason for each MR. Risky MRs get the right people automatically suggested. Low-risk MRs move faster. The reviewer is still the one who decides.

PRD-lite

User problem. Reviewers cannot tell which MRs are risky without opening each one, so they triage by age or size.

Core workflow

1. Developer opens or updates an MR.
2. The assistant extracts features from the diff: paths touched, size, test changes, risky patterns, dependency and config changes.
3. Deterministic checks run first (secrets, privileged containers, open firewall, injection patterns). These cannot be "outvoted" by the model.
4. The risk model gives a score 0-100 and a band.
5. The LLM writes a 2-4 sentence explanation from the signals and the diff.
6. The result appears in the MR and in the reviewer's queue, sorted by risk.
7. The reviewer accepts the band or overrides it with a reason.
8. After merge, outcome signals (revert, hotfix within 14 days, linked incident) are collected for evaluation and retraining.

AI role

- *ML model:* risk scoring, learned from the team's own MR history.
- *LLM:* explanation and review suggestions in plain language.
- *Not AI:* secret/unsafe-config detection and routing rules. These are deterministic on purpose.

Requirements (MVP)

#	Requirement	Notes
R1	Score every MR within 2 minutes of opening / update	Must not slow down the developer
R2	Show band + top 3 reasons + suggested review path	Reasons must point to files / lines
R3	Secret or unsafe-config hit always forces High	No model can lower it
R4	Reviewer can override with a mandatory reason	Stored for learning and audit
R5	Mark scores close to a band threshold as "uncertain"	Avoid false precision
R6	Never block or auto-approve a merge	Advisory only
R7	Do not use author identity / seniority as a feature	Fairness and trust
R8	Log score, signals, decision and outcome per MR	For evaluation and audit

Data assumptions

- We can get at least 6-12 months of MR history from the Git platform API.
- We can label "problem MRs" from reverts, hotfix MRs that reference them, and incident tickets linked to a commit. This labelling is noisy and incomplete (many problems are never linked back).
- Diff content may be sent to an LLM only if the company's data policy allows it.

Risks (detail in 07)

- False alarms cause alert fatigue and people stop looking.
- Misses create false confidence ("the tool said Low").
- Code confidentiality with an external LLM.
- Gaming: splitting changes to stay under the radar.

Fallback logic

Situation	Behaviour
Model or LLM unavailable	Show rule-based score only, clearly labelled "rules only"
Score near a threshold	Show band as uncertain, suggest the reviewer decide
Diff too large to analyse	Mark as "not analysed, needs manual triage", never silently Low
Unknown file types / new folders	Fall back to rules, flag "low confidence"

Feature prioritisation (RICE)

Reach = MRs per month affected (pilot team ~150). Impact on a 0.5-3 scale. Confidence %. Effort in person-weeks.

Feature	Reach	Impact	Confidence	Effort	RICE	Priority
Rule-based risk score + reasons	150	2	80%	2	120	MVP
Secret / unsafe-config detection (reuse scanners)	150	3	90%	1	405	MVP
Override with reason + decision log	150	1.5	80%	1	180	MVP
Risk-sorted queue view	150	2	70%	1.5	140	MVP
LLM explanation	150	1	60%	2	45	Pilot
Learned ML score	150	2	50%	4	37.5	Pilot
Manager dashboard	20	1	70%	2	7	Later
Auto reviewer assignment	150	1	40%	3	20	Later

The secret detection scores highest because it is cheap (reuse) and the impact of a miss is severe. The learned model scores lower than I expected. That is mostly the low confidence: we don't yet know if the labels from MR history are good enough. That is exactly what the pilot needs to find out.

MVP definition

The MVP proves one thing: risk-sorted triage with reasons helps reviewers put attention on the right MRs, and they trust it enough to use it.

In scope: rule-based score, deterministic security checks, reasons, suggested review path, override with reason, decision log, risk-sorted queue. Out of scope: learned model, LLM text, auto-assignment, dashboards, blocking.

The prototype in this repo is this MVP, running on synthetic MRs.

Roadmap with proof gates

Phase	Duration	What happens	Gate to move on
0. Prototype	done	Clickable prototype, synthetic MRs, rules engine	Workflow makes sense to 3-5 reviewers I show it to
1. Shadow pilot	2 weeks	Score real MRs in 2-3 repos, log only, not shown to reviewers	On past problem MRs, recall clearly better than the size rule; flags under ~30% of queue
2. Advisory pilot	6 weeks	Show score + reasons in MR, collect overrides	Override rate < 25%; reviewers say it's useful (survey); no "missed High" incident
3. Add ML + LLM	4-6 weeks	Train model on labelled history, add LLM explanation	Model beats rules on held-out data; explanation rated accurate in > 80% of spot checks
4. Scale to more teams	later	Per-team calibration, dashboard	Same gates hold for a second team

The difference from a normal software roadmap is that each phase has a *learning* gate, not just a delivery date. If a gate fails, we stop or fix, we don't push ahead.

Success metrics

Type	Metric	Target for pilot
Product	% of MRs where the reviewer opened the risk panel	> 60%
Product	Reviewer usefulness rating (1-5)	≥ 3.5
Business	Recall of problem MRs in High band (vs size rule)	Clearly higher than baseline, aim > 60%
Business	Median time to first review for Low-risk MRs	Goes down
Business	Problem MRs that reached production without senior/security review	Goes down
Trust / guardrail	Override rate	< 25%
Trust / guardrail	Share of queue flagged High	< 30% (alert fatigue guard)
Trust / guardrail	High-severity incident from an MR the tool rated Low	0 in pilot, every case reviewed

How these artifacts change because it is an AI product

Artifact	Normal version	What changed here
Problem statement	"Reviews are slow"	Framed around a *decision*: how much review an MR needs, and from whom
User story	"As a reviewer I want a list"	Includes error cost: what happens when the score is wrong and how the user notices
PRD	Features and acceptance criteria	Added data assumptions, labelling source, fallback behaviour, things the model must never decide
Prioritisation	Value vs effort	Confidence in the data is a real factor. The learned model drops in priority because the labels are uncertain
Roadmap	Dates and releases	Proof gates. Shadow mode before showing anything to users
Metrics	Usage, time saved	Model metric (recall) tied to business metric (problem MRs caught) and a guardrail (queue share, override rate)
Launch checklist	QA sign-off	Plus: rollback to rules-only, override path, audit log, no blocking

04 UX, Workflow and Trust Design

Design principle

It should feel like a colleague who glanced at the MR and said "careful, this touches auth and there are no tests". It should not feel like a gatekeeper. If developers feel judged or blocked, they will route around it.

Main workflow

```
Developer opens MR
      |
      v
Assistant analyses diff (rules + scanners, later ML + LLM)
      |
      v
Risk band + reasons + suggested review path
      |                          |
      v                          v
Shown in MR (comment/panel)   Reviewer queue sorted by risk
      |
      v
Reviewer: Accept  or  Override (reason required)
      |
      v
Decision logged  ->  after merge, outcome collected (revert / hotfix / incident)
      |
      v
Used for evaluation and retraining
```

Key user journey (reviewer)

1. Priya, a tech lead, has 18 open MRs on Monday morning.
2. She opens the review queue. It is sorted by risk instead of age. The top 3 are High: a Terraform change opening a firewall, an MR with what looks like an AWS key, and a payment rounding fix with no tests.
3. She opens the Terraform one. The panel says *"Unsafe infra setting: `source_ranges = ["0.0.0.0/0"]` in infra/terraform/db.tf. Suggested: platform review."* She tags the platform team.
4. The payment fix is marked High but "uncertain" (score close to the threshold). She looks, agrees it needs tests, and accepts.
5. A UI MR under a `payments/` folder is also High. She sees it only adds icons, overrides to Low with the reason "UI only, no logic". That override is logged.
6. The remaining Low MRs (README, translations, test-only) she approves quickly or leaves to a junior reviewer.

Screens (prototype)

The prototype has these screens. Open `prototype/index.html` to see them.

Screen	Purpose
Review queue	All open MRs sorted by score. Shows band, size and status
MR detail panel	Score bar, uncertainty warning, summary, signals with points, suggested review path, files
Reviewer decision	Accept, or override to another band with a mandatory reason
Decision log	All decisions with override rate. Export as JSON
Analyse a diff	Paste any `git diff` and see the analysis. Useful for demo and for developers to self-check before opening an MR
Evaluation	Baseline vs prototype on the synthetic set, with per-MR outcomes

Text wireframe of the detail panel:

```
+-------------------------------------------------------------+
| MR-114 | Fix rounding in invoice total                      |
| [High risk] score 45/100   [====================]          |
| ! Close to a band threshold. Treat the band as uncertain.   |
|                                                             |
| SUMMARY                                                     |
| Rated high mainly because of: payments / billing code;      |
| code changed, no tests changed; sensitive area without tests.|
|                                                             |
| WHY                                                         |
|  - Payments / billing code            +20  src/payments/... |
|  - Code changed, no tests changed     +15                   |
|  - Sensitive area changed w/o tests   +10                   |
|                                                             |
| SUGGESTED REVIEW PATH                                       |
|  - Senior developer review                                  |
|  - Add or update tests                                      |
|  - Do not merge until the above reviews are done            |
|                                                             |
| [Accept High]   [Low/Med/High v] [reason........] [Override] |
+-------------------------------------------------------------+
```

Trust and explainability

What the user needs to know

- The band, and the 2-3 main reasons, each pointing to a file or line.
- What to do about it (review path), not just a number.
- That it is advisory. The reviewer decides.

What must be visible

- Every signal with its points, so the score is never a black box.
- When the score is uncertain (near a threshold).
- When the analysis is degraded (rules only, diff too big, unknown files).

What should be hidden

- Raw model probabilities with many decimals. They suggest precision we don't have.
- Any per-developer "risk history". It is not used, and it should not appear anywhere.
- Secret values themselves. Show the file and line, mask the value.

Uncertainty handling

Case	Behaviour
High confidence (score clearly in a band)	Show band and reasons normally
Low confidence (within 4 points of a threshold)	Show band with "uncertain" warning, ask reviewer to decide
Hard rule hit (secret, privileged container, open firewall)	Always High, regardless of model. Explain the exact line
Failure / fallback	Model down: rules-only score, labelled. Diff too large or unparseable: "Not analysed, manual triage", never shown as Low

Human-in-the-loop

- **Every** band can be overridden, up or down, by the reviewer.
- Overrides need a short reason. This is a bit of friction on purpose: it stops blind clicking and gives us learning data.
- The tool never approves, merges or blocks.
- Security-relevant hard-rule hits (secrets) also notify the security channel, because those need action even if the MR is closed.
- Override reasons are reviewed every two weeks during the pilot. If the same override keeps appearing (like "UI only under payments folder"), the rule is fixed.

05 Data, Model and Evaluation Strategy

Problem as AI task

There are three different tasks here, and treating them separately keeps things honest:

Task	Type	Approach
Is there a secret / unsafe setting / injection pattern?	Detection	**Deterministic rules and existing scanners.** Not ML. These must be predictable
How risky is this MR overall?	**Classification / ranking** (Low / Medium / High, or a 0-100 score used to rank the queue)	Rules in MVP, then a learned model (logistic regression or gradient boosting) on MR history
What should the reviewer know?	**Generation**	LLM, given the diff and the signals, writes a short explanation

Data sources

Internal (main source)

- MR metadata from the Git platform API: files changed, lines added/removed, labels, approvals, review comments, time to merge.
- Diffs (file paths and added lines).
- Revert commits and hotfix MRs that reference an earlier MR.
- Incident / bug tickets linked to commits or MRs.
- CI results: test pass/fail, scanner findings.

External

- Rule packs from open-source secret and SAST scanners.

User-generated (from the product itself)

- Reviewer overrides and reasons. These become the most useful data over time, because they are direct human labels.

Not used: author identity, seniority, team, past "bad MR" counts. See 07.

Labelling / ground truth

The label is "did this MR cause a problem after merge?", defined as any of:

- reverted within 30 days,
- a hotfix MR within 14 days that references it or touches the same lines,
- linked to an incident ticket,
- a security finding traced back to it.

Honest problems with this label

- It under-counts. Many problems are fixed quietly without linking back to the original MR.
- It mixes severities. A typo revert and a production outage both count as 1. In the pilot I would weight by severity where incident data exists.
- Rare positives. Maybe 3-8% of MRs, so accuracy is a useless metric. A model that always says "fine" would be 95% accurate.

Reviewer overrides during the pilot give a second, cleaner source of labels.

Collecting the history for the shadow pilot. `scripts/collect_mr_history.py` pulls merged MRs from the GitLab API and writes one CSV row per MR: files, lines added/removed, test files touched, paths, and a `problem` label when a later revert or hotfix MR references it (`!123`). It needs only a read-only token and the Python standard library. Author fields are deliberately not exported. The first thing to check with real output is how many problem labels we actually get. If it is only a handful in 6 months, the learned model has to wait and the pilot runs on rules plus reviewer overrides.

Baseline

The baseline is what teams already do informally: **look harder at big MRs (more than 300 changed lines).** Any model has to clearly beat this, otherwise it is not worth the complexity.

A second baseline for the pilot: the rules-only version (what the prototype runs). The learned model must beat the rules, not just the size rule.

Model / API choice

- **MVP:** weighted rules. Transparent, no training data needed, easy to explain. Every signal and its points are shown.
- **Pilot:** logistic regression or gradient boosted trees on the same features plus a few more (file churn history, how many services import the changed file). I prefer something simple and explainable over a deep model. Per-feature contributions (or SHAP values for trees) map directly to the "Why" list in the UI.
- **LLM:** hosted API for the explanation, *or* a self-hosted model if code cannot leave the network. The LLM gets the signals as input and is told to explain only those. It does not produce the score. This limits hallucinated reasons.

Prototype evaluation (synthetic data)

I wrote 24 synthetic MRs covering typical cases: docs, UI refactors, auth, payments, migrations, k8s, terraform, CI, dependency bumps, and a few traps. Each has an "after merge" outcome I assigned based on what typically happens with that kind of change. 11 of 24 are problem MRs (much higher than real life, on purpose, to have enough cases).

Results, computed live in the prototype's Evaluation tab:

	Size baseline (> 300 lines)	Prototype (High band)
MRs flagged	5 of 24 (21%)	9 of 24 (38%)
Problem MRs caught (recall)	1 of 11 (9%)	8 of 11 (73%)
Flags that were real (precision)	1 of 5 (20%)	8 of 9 (89%)
False alarms	4 (UI refactor, translations, export endpoint, lockfile bump)	1 (MR-122, payment icons)
Missed problem MRs	10	3 (MR-121, MR-123, MR-124)

What the misses teach

- **MR-121, date utils refactor.** Small and tested, but used by 6 services. The tool has no idea about blast radius. This is the biggest gap, and it's why "how many places import this file" is on the pilot feature list.
- **MR-123, cache TTL change.** A 2-line config change with a business impact (stale prices). No rule can know that. Probably only reviewer judgement or incident history can.
- **MR-124, auth library upgrade.** Dependency bumps get a small score. A dependency used for auth should probably count as auth. That is an easy rule fix.

What the false alarm teaches

- **MR-122, payment icons.** The path contains `payments`, so it looks like payment logic. Path keywords over-trigger. A reviewer would override this, and that override is exactly the signal to fix the rule (e.g. ignore UI-only files for the payments rule).

The big caveat: I designed both the rules and the data, so the prototype is bound to do well. These numbers show the mechanism works. They are not evidence it will work on a real codebase. The shadow pilot is where that gets tested.

Evaluation framework for the pilot

Metric type	Metric	Why it matters
Technical	Recall of problem MRs in the High band	The whole point is not missing risky MRs
Technical	Precision of the High band	Too many false alarms = people ignore it
Technical	Recall at a fixed queue share (e.g. top 20% of MRs by score)	Fair way to compare with baseline at equal reviewer effort
Technical	LLM explanation accuracy (spot check 30 per week: is every stated reason actually in the diff?)	Catch hallucinated reasons early
Business	Problem MRs that reached production without senior/security review	The outcome we actually care about
Business	Median time-to-first-review for Low MRs	Low-risk work should move faster
Business	Senior reviewer hours spent on Low MRs	Is effort shifting to where it matters?
Trust / guardrail	Override rate (and direction: up vs down)	Rising override rate = model drifting or not trusted
Trust / guardrail	Share of queue flagged High	Alert fatigue guard
Trust / guardrail	Any High-severity incident from an MR rated Low	Every case reviewed

Launch / go-no-go thresholds

Move from shadow to advisory pilot if:

- recall of past problem MRs in High is clearly above the size baseline (aim > 60%), and
- High covers under ~30% of the queue, and
- no hard-rule (secret/unsafe) case was missed.

Move from advisory pilot to wider rollout if:

- override rate stays under 25% over 6 weeks,
- reviewer usefulness rating is 3.5 or more out of 5,
- explanation spot checks are accurate in more than 80% of cases,
- no serious incident came from an MR the tool rated Low without a reviewer override.

Stop if: reviewers stop opening the panel, override rate keeps rising, or the label data turns out too sparse to evaluate anything.

06 Business, Economics and Scaling

All numbers below are **assumptions** for a mid-sized product company, used to check whether the idea can make sense. None of them are measured. The pilot is where they get replaced with real numbers.

Monetization model

Primary: internal value case. This is built for an internal engineering org. Nobody pays per MR. The question is whether it saves more engineering and incident cost than it costs to run.

Later option: SaaS. If it works well across several teams, it could be sold as a per-active-developer subscription (like most dev tools), with a self-hosted option for companies that cannot send code outside. I would not plan for this now. It is a different business with competitors already in the AI code review space.

Value logic

The product creates value in three ways:

1. **Fewer escaped problems.** Risky MRs get the right reviewer before merge, so fewer reverts, hotfixes and incidents.
2. **Faster flow for low-risk MRs.** They can be reviewed faster or by a wider set of reviewers, so lead time drops.
3. **Better use of senior and security people.** Their time moves from "look at everything" to "look at what is flagged".

Assumed org for the numbers

Assumption	Value
Engineers	60
MRs per month	~600
Average review time per MR	~30 min
Loaded cost of a senior engineer hour	~₹2,000
Problem MRs that cause a real incident per month	~3
Average cost of one incident (engineer time, firefighting, customer impact)	~₹1.5 lakh

Cost drivers

Cost	Estimate per month	Notes
LLM API	~₹3,000-₹10,000	~600 MRs x one call each, a few thousand tokens per call. Depends heavily on model and diff size. Cap diff size sent
Compute for scoring + scanners	Small, runs in existing CI	Mostly already paid for
Build effort (one-off)	~8-10 person-weeks	Integration, scoring, UI in MR, logging
Maintenance	~1-2 days per month	Rule tuning from overrides, retraining
Review cost (human)	Goes *up* for High MRs, *down* for Low	This is intended. It is a shift, not just a saving
Privacy / security review of the tool itself	One-off	Especially if code goes to an external API

Unit economics view

Rough monthly view with the assumptions above:

- **Saving on low-risk reviews:** if 50% of MRs are Low and their review time drops from 30 to 20 min, that is 300 x 10 min = 50 hours, about ₹1 lakh.
- **Extra cost on high-risk reviews:** if 20% are High and get +20 min of senior/security time, that is 120 x 20 min = 40 hours, about ₹80,000.
- So the review time roughly evens out. **The real case is incidents:** if the tool helps prevent even 1 of the ~3 monthly incidents, that's about ₹1.5 lakh saved, against a running cost of maybe ₹10,000-₹20,000 a month.

The honest conclusion is that this is **not a "save reviewer time" product.** It is a risk-reduction product that is roughly neutral on review time. The pitch to management should be "fewer incidents for about the same review effort", not "faster reviews". If the pilot shows it doesn't reduce escaped problems, the business case is weak even if reviewers like it.

Scaling risks

Risk	Why it matters	Mitigation
Latency	If the score appears 10 minutes after the MR, reviewers have already started	Run rules first (seconds), add LLM text when ready
Cost growth	LLM cost grows with MR count and diff size	Only call the LLM for Medium/High, cap tokens, cache by commit
Workflow complexity	Each team has different folders, conventions, risk areas	Per-repo config for sensitive paths, team-level calibration
Governance burden	Logs contain code snippets and possibly secrets	Mask secrets, retention limits, access control on logs
Model drift	New frameworks, folder moves, new team habits	Monitor override rate, retrain quarterly, rules as fallback
Alert fatigue at scale	More teams = more noise if not calibrated	Keep the High share target per team, not global

Operations implications

- Someone has to own the rules and the model. During the pilot that's me. At scale it would sit with a platform / developer experience team.
- Security team needs a clear channel for hard-rule hits like leaked secrets.
- Engineering managers need to agree that the tool is advisory, and that "the tool said Low" is never an excuse in a post-mortem.

07 Ethics, Governance and Risk Note

Main risks

1. **False sense of safety.** A "Low" label makes reviewers relax, and a risky MR goes through *because* the tool rated it low. This is the most serious risk. It's worse than not having the tool.
2. **Alert fatigue.** Too many High flags, and people stop reading them. Then the tool adds noise and nothing else.
3. **Scoring people instead of changes.** If author history is used, or if scores are visible in a way managers start using in performance reviews, it becomes a surveillance tool.
4. **Code confidentiality.** Sending diffs to an external LLM may break company policy or customer contracts. Diffs sometimes contain secrets or customer data.
5. **Hallucinated explanations.** The LLM states a reason that isn't in the diff ("this changes the password hashing") and the reviewer trusts it.
6. **Gaming.** Developers learn the rules and split or rename things to avoid being flagged.

Bias

- **Author-based bias.** Junior or new developers statistically have more reverted MRs, partly because they get the harder onboarding tasks or less context. A model that learns from author history would flag them more, regardless of the actual change. I removed author identity, seniority and team from the features entirely. The score is about the change, not the person.
- **Folder / team bias.** Teams working on legacy or critical systems will always score higher. That is correct for review routing, but it must not be read as "this team writes worse code". Dashboards (later) should show risk by *area*, not rank teams.
- **Label bias.** Problems are linked back to MRs more often in some teams than others (better incident discipline). The model may learn "team X causes incidents" when actually team X just records them better. Check label coverage per team before training.

Privacy

- Default: the scoring and the scanners run inside our own CI. No code leaves.
- LLM explanation: only if approved by security. Otherwise use a self-hosted model, or send only the signal summary (file names, signal labels), not the diff.
- Secrets found are **masked** in all logs and UI. Only the file and line are shown.
- Decision logs have a retention limit and access is restricted to the pilot team and security.

Hallucination / unsafe output

- The LLM never produces the score. It only explains signals that the rules / model already found.
- The prompt passes those signals in and says to explain only them, and to quote file paths exactly.
- Spot check 30 explanations a week in the pilot. If more than 20% have an unsupported claim, turn the LLM text off and fall back to template text (what the prototype does today).

Misuse risk

Misuse	Control
Managers use scores to judge developers	Scores are per MR only. No per-person aggregation is built. Written policy that scores are not for performance reviews
"The tool said Low" used as an excuse after an incident	Advisory only. Reviewer remains accountable, stated in the team's review guidelines
Gaming the rules	Rules are not secret, but the learned model and hard rules (secrets etc.) are hard to dodge. Watch for unusual MR splitting patterns
Using it to auto-merge Low MRs	Explicitly out of scope. Would need a separate decision with much stronger evidence

Controls

- **Guardrails:** hard rules (secrets, privileged containers, open firewall, injection patterns) always force High; the model cannot lower them. Diffs that can't be analysed are shown as "not analysed", never Low.
- **Review process:** every band can be overridden with a reason. Overrides reviewed every two weeks.
- **Escalation path:** secret hits go to the security channel automatically. A Low-rated MR linked to an incident triggers a review of the rules.
- **Logging / auditability:** for every MR, store score, signals, rule/model version, reviewer decision and reason, and later the outcome. That way we can always answer "why did the tool say this, and who decided what".
- **Rollback:** one switch to go back to rules-only, and one to turn the whole thing off without affecting the merge process.

Launch blockers

I would pause or not launch if any of these happen:

- A hard-rule case (e.g. a real secret) is missed in shadow mode.
- The High band flags more than ~30% of the queue and can't be tuned down without losing most of the recall.
- Security does not approve the data flow for the LLM and there is no acceptable self-hosted option. In that case: launch without LLM text, rules + model only.
- Any sign that scores are being used to evaluate individual developers.
- In the advisory pilot, a serious incident comes from an MR the tool rated Low and the reviewer relied on that rating. That needs a full review before continuing.

08 Prototype / MVP

Prototype type

Offline single-page web app: `prototype/index.html`. Open it in any modern browser. No install, no server, no network calls, no login.

How to run

1. Download or clone this repository.
2. Open `prototype/index.html` in Chrome, Edge, Firefox or Safari.
3. That's it.

Demo script (about 5 minutes)

1. **Review queue tab.** Point out that the queue is sorted by risk, not by age. The top of the list is the secret leak, the SQL injection and the migration. The README fix and translations are at the bottom.
2. Click **MR-104** (S3 config). Show the hard-rule hit: "Possible secret in added lines". Point out it is High regardless of size (only 6 lines).
3. Click **MR-114** (invoice rounding). Show the "uncertain" warning because the score is right at the threshold. Accept it.
4. Click **MR-122** (payment icons). It's High only because the path has `payments`. Override it to Low with the reason "UI only, no logic". Show the decision log and override rate updating.
5. **Analyse a diff tab.** Click "Load a sample diff", then Analyse. It picks up the hardcoded login bypass area and the API key.
6. **Evaluation tab.** Show baseline vs prototype. Then scroll to the misses (MR-121, 123, 124) and explain why the tool can't see them. Read the caveat box out loud.
7. **How it works tab.** Show the full rule table and the "deliberately not used" section.

What it demonstrates

- The core decision loop: MR comes in, gets a score, band and reasons, reviewer accepts or overrides.
- That reasons can be specific (file, line) and not just a number.
- Uncertainty handling (borderline scores) and hard rules that can't be outvoted.
- Human override with reason capture, and the override rate as a live guardrail metric.
- A fair comparison against the obvious baseline, including the cases where the prototype fails.

AI role

In the prototype:

- The **risk score** is a weighted rule set. It stands in for the ML model I would train in the pilot, and it will remain as the baseline and fallback.
- The **summary** text is generated from templates based on the signals found. It stands in for the LLM explanation.

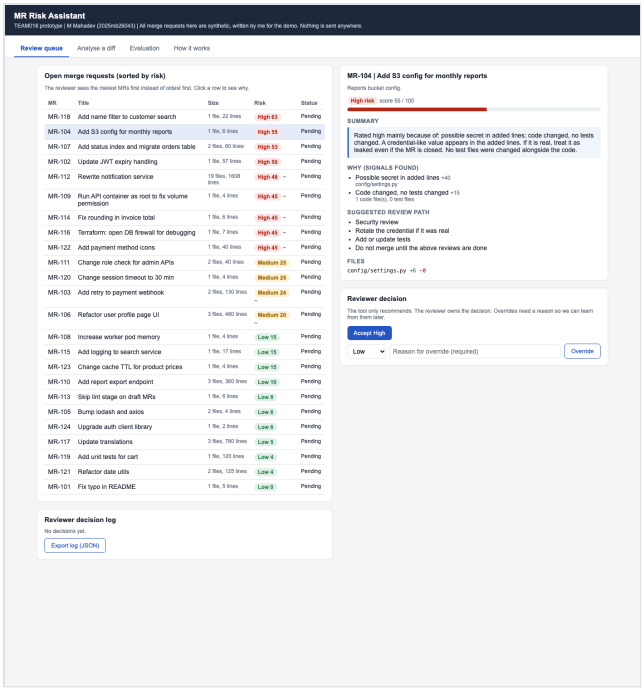
I chose not to call a real LLM in the prototype for two reasons: it would need an API key in a public repo, and sending sample code to an external service is the exact privacy question the pilot has to settle first. The interaction (score, reasons, explanation, override) is what the prototype needs to prove, and that works the same either way.

What it does not yet prove

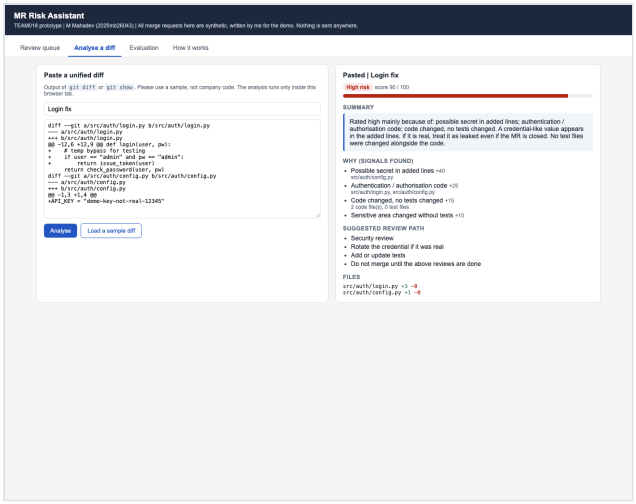
- **Real-world accuracy.** I wrote both the rules and the 24 sample MRs, so the evaluation numbers are optimistic by design.
- That a learned model beats the rules. There is no trained model yet.
- That an LLM explanation is accurate and actually helps reviewers.
- Integration with GitLab / GitHub (showing the panel inside the MR, webhooks).
- That reviewers will trust and use it over weeks, not just in a demo.
- Cost and latency at real volume.

Screenshots

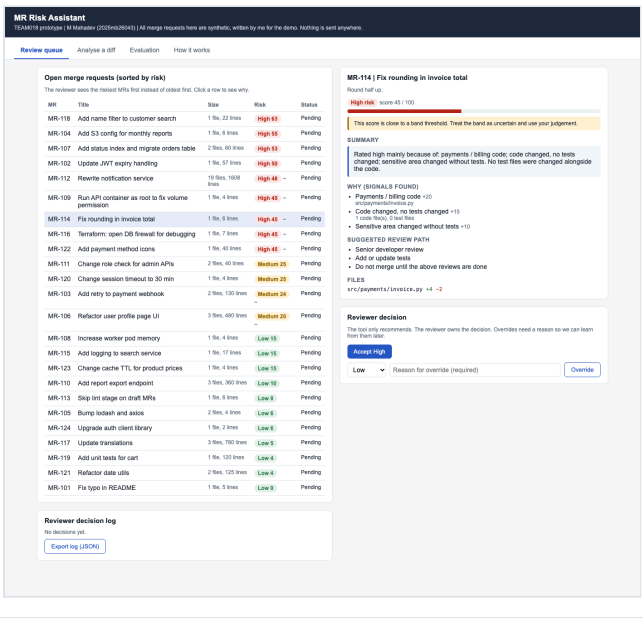
Review queue, sorted by risk



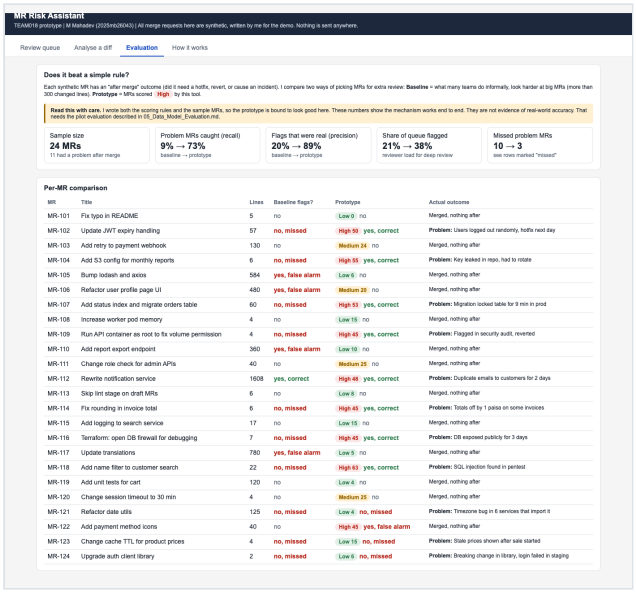
Analyse a pasted diff



Borderline score with uncertainty warning



Evaluation against the size baseline



Links

- Repository: https://github.com/MAIP2026July/TEAM018
- Prototype: `prototype/index.html` (open locally; can also be served with GitHub Pages)
- Screenshots: `Appendix/Screenshots/`